AI Release Intelligence Assistant

Асистент релізів і тестування для команд Microsoft Power Platform

Inna Kurylo · Power Platform Development Lead · AI Engineering, 2 потік · галузь: IT / розробка — баг-репорти, release notes

40	20 + 20	10	10 / 3
«брудних» текстів для кейсу 1	реальні GitHub + синтетичні	текстів у golden set	полів у схемі / enum

ІДЕЯ ПРОЄКТУ

Команди на Power Platform живуть у постійному потоці змін: щомісячні оновлення платформи, Dataverse, Power Automate, pac CLI, CoE Starter Kit — і власні релізи рішення. Інформація розкидана по release notes, баг-репортах в Azure DevOps, Teams і листах, написана різними мовами й у різному стилі. Щоб відповісти на питання **«що зачепить оновлення і що перетестувати?»**, Release/QA lead годинами читає документи вручну — і щось пропускає, звідси регресії на проді.

Система структурує сирі баг-репорти, шукає по релізній документації з посиланнями на джерела і формує рекомендації для оновлення й регресійного тестування.

ДЛЯ КОГО

- **Release Manager / Dev Lead** — зміни в релізі, breaking changes, ризики
- **QA Lead** — які тест-кейси запускати, які дефекти можуть повернутися
- **Розробник** — чи зачіпає нова версія pac CLI / платформи наші flows, плагіни, PCF

ДАНІ

Лише публічні та синтетичні, без корпоративних даних.

- 20 баг-репортів з GitHub (`microsoft/coe-starter-kit`, `microsoft/powerplatform-build-tools`), знеособлені скриптом
- 20 синтетичних баг-репортів вигаданої компанії Contoso: UA / EN / суржик, пропущені поля, не-дефекти
- Для пошуку (етап 2): GitHub Releases, Microsoft release notes, синтетичні тест-кейси й user stories

4 ЕТАПИ

1. **Extraction** — сирий баг-репорт → JSON за схемою `DefectReport`; порівняння провайдерської й локальної моделей.
2. **RAG** — пошук по release notes, дефектах і тест-кейсах; відповідь з посиланням на джерело.
3. **Агент** — за новим релізом знаходить зачеплені компоненти, минулі дефекти й тест-кейси → регресійний тест-план.
4. **Сервіс** — FastAPI, автентифікація, моніторинг, облік вартості; виклик з Power Automate через custom connector.

КЕЙС 1 — STRUCTURED EXTRACTION

Схема	`DefectReport`: title, is_defect, **components[{type, name}]**, environments, severity, release_version, steps_to_reproduce, expected/actual_result, is_regression. Enum: тип компонента (12 значень), середовище, серйозність.
Моделі	Провайдерська: `openai/gpt-4o-mini` через OpenRouter, strict structured output. Локальна: `qwen2.5:7b-instruct` через Ollama. Спільний інтерфейс `LLMClient`.
Семплінг	A: temperature 0, top_p 1 · B: temperature 0.7, top_p 0.9
Метрики	Точність по полях на golden set, % валідних (ok / repaired / failed), latency p50/p95, вартість на 1000 документів.
Надійність	Retry 2× з backoff, timeout 60 с; невалідна відповідь → 1 спроба виправлення → failed; лог моделі, версії промпту й параметрів у кожному записі.
Стан	Готово: дані, схема, промпт v1, пайплайн, тести. Далі: прогони, golden set, метрики, промпт v2, REPORT.md.

> **Питання:** 1) чи підходить тема і поєднання публічних та синтетичних даних? 2) чи нормально, що в кейсі 1 витягую лише баг-репорти, а release notes підключаю з етапу RAG? 3) чи підходить пара моделей gpt-4o-mini + qwen2.5:7b?